Fingertip Trajectory Recognition

1. Introduction

MediaPipe is a data stream processing machine learning application development framework developed and open-sourced by Google. It is a graph-based data processing pipeline used to build applications that use various forms of data sources such as video, audio, sensor data, and any time series data.

MediaPipe is cross-platform and can run on embedded platforms (Jetson nano, etc.), mobile devices (iOS and Android), workstations and servers, and supports mobile GPU acceleration. MediaPipe provides cross-platform, customizable ML solutions for real-time and streaming media.

The core framework of MediaPipe is implemented in C++ and provides support for languages such as Java and Objective C. The main concepts of MediaPipe include Packet, Stream, Calculator, Graph, and Subgraph.

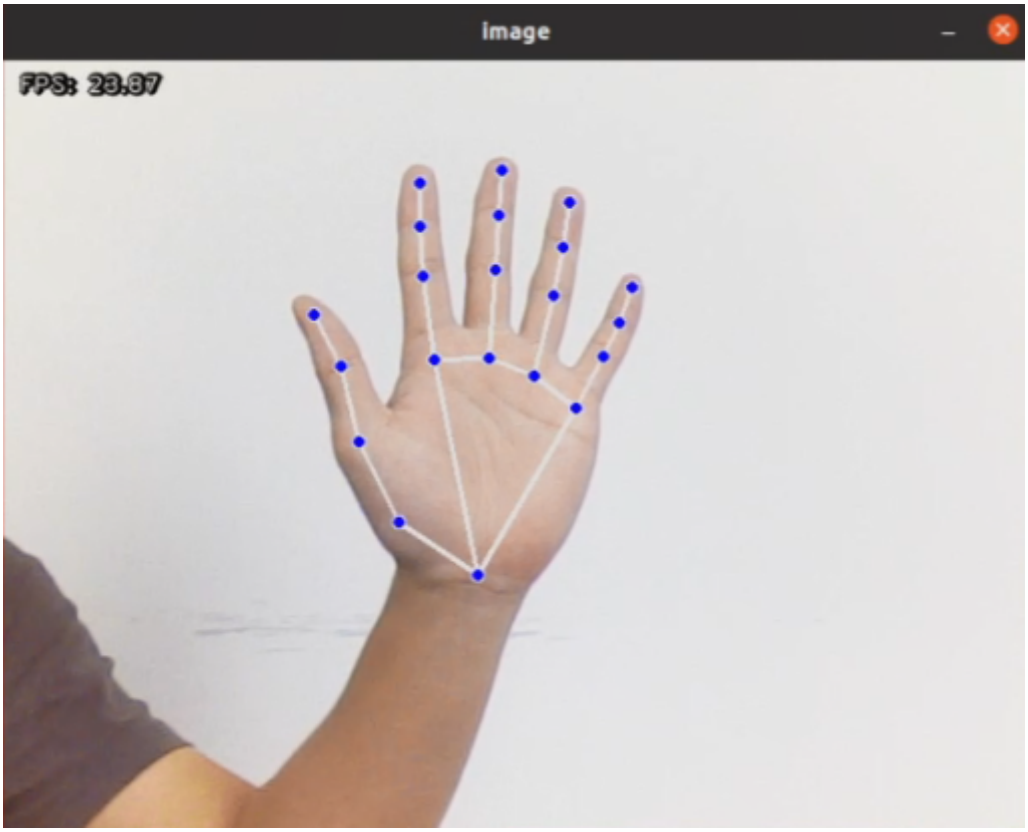
Features of MediaPipe:

- End-to-end acceleration: Built-in fast ML inference and processing accelerates even on ordinary hardware.
- Build once, deploy anywhere: Unified solution for Android, iOS, desktop/cloud, web and IoT.
- Ready-to-use solutions: Cutting-edge ML solutions that showcase the full capabilities of the framework.
- Free and open source: Framework and solutions under Apache2.0, fully scalable and customizable.

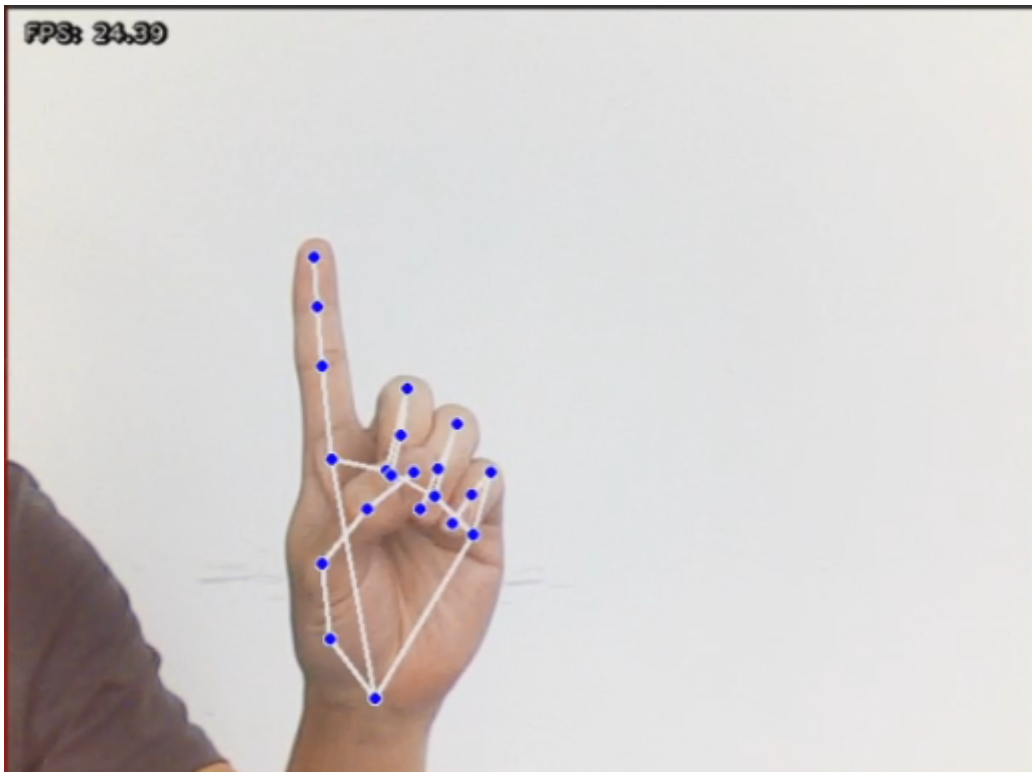
2. Launch

2.1. Program Description

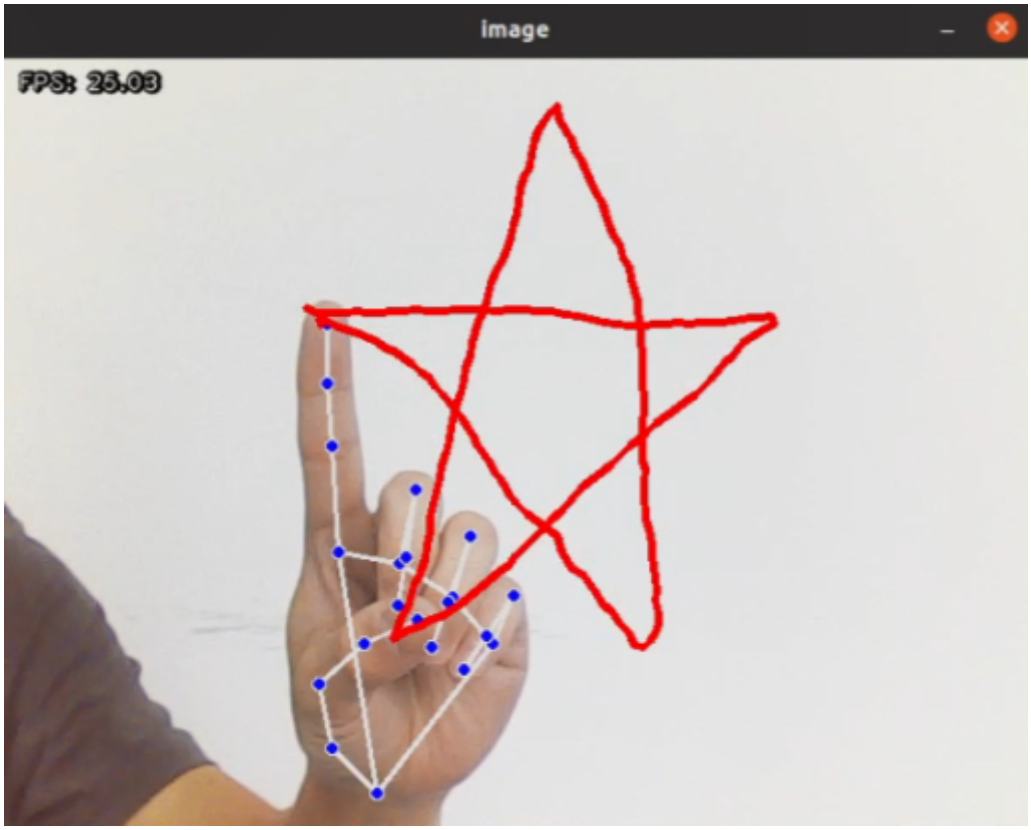
After the program starts, the camera captures images. Place your hand flat in the camera frame, with fingers spread open and palm facing the camera, similar to the number 5 gesture. The image will draw all the joints on the entire hand. Adjust the hand position to be in the upper middle part of the screen.



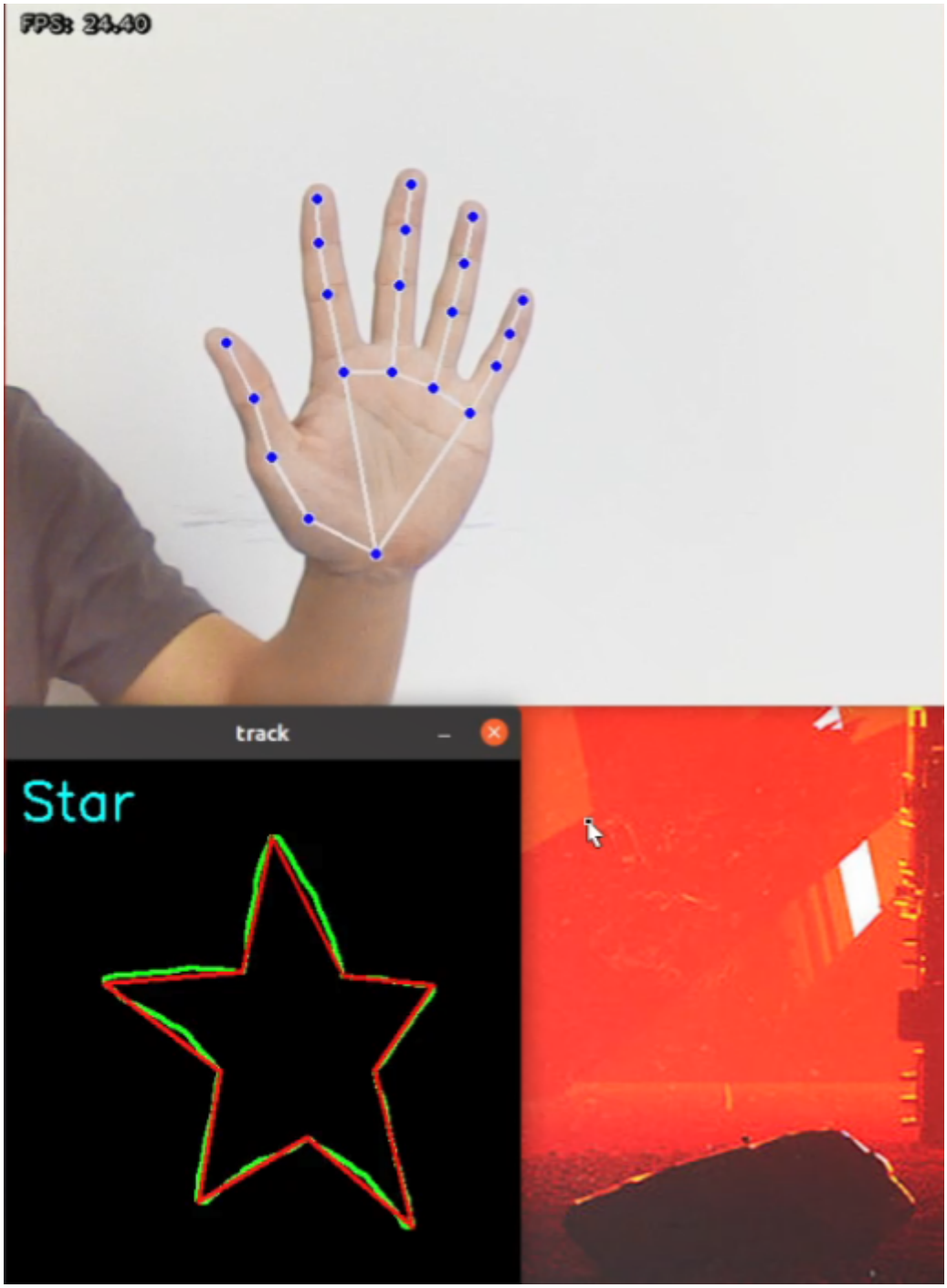
At this point, keep the index finger unchanged while retracting other fingers, similar to the number 1 gesture.



While maintaining the gesture 1 posture, move the finger position, and red lines will appear on the screen, drawing the path of the index finger movement.



When the drawing is complete, open all fingers, similar to the number 5 gesture, and the drawn shape will be generated at the bottom.



Note: The drawn shape needs to be in a closed state, otherwise some content may be missing.

Currently, there are four recognizable trajectory shapes: triangle, rectangle, circle, and five-pointed star.

2.2. Program Launch

- Enter the following command to start the program

```
ros2 run dofbot_mediapipe 15_FingerTrajectory
```

Press the q key in the image or press Ctrl+c in the terminal to exit the program.

3. Source Code

Code path:

```
#  need to enter the docker container to view
~/dofbot_ws/src/dofbot_mediapipe/dofbot_mediapipe/15_FingerTrajectory.py
```

```
#!/usr/bin/env python3
# coding: utf8
import os
import enum
import cv2
import time
import numpy as np
import mediapipe as mp
import rclpy
from rclpy.node import Node
import queue
from sensor_msgs.msg import Image
from dofbot_utils.fps import FPS
import gc
from dofbot_utils.vutils import distance, vector_2d_angle, get_area_max_contour
from cv_bridge import CvBridge
from Arm_Lib import Arm_Device
Arm = Arm_Device()
init_joints = [90.0, 150.0, 10.0, 10.0, 90.0, 0.0]
Arm.Arm_serial_servo_write6_array(init_joints,2000)

def get_hand_landmarks(img, landmarks):
    """
    Convert landmarks from MediaPipe's normalized output to pixel coordinates
    :param img: The image corresponding to pixel coordinates
    :param landmarks: Normalized key points
    :return:
    """
    h, w, _ = img.shape
    landmarks = [(lm.x * w, lm.y * h) for lm in landmarks]
    return np.array(landmarks)


def hand_angle(landmarks):
    """
    Calculate the bending angle of each finger
    :param landmarks: Hand key points
    :return: Angles of each finger
    """
    angle_list = []
    # thumb 大拇指
    angle_ = vector_2d_angle(landmarks[3] - landmarks[4], landmarks[0] -
landmarks[2])
    angle_list.append(angle_)
    # index 食指
    angle_ = vector_2d_angle(landmarks[0] - landmarks[6], landmarks[7] -
landmarks[8])
    angle_list.append(angle_)
    # middle 中指
```

```
    angle_ = vector_2d_angle(landmarks[0] - landmarks[10], landmarks[11] -
landmarks[12])
    angle_list.append(angle_)
    # ring 无名指
    angle_ = vector_2d_angle(landmarks[0] - landmarks[14], landmarks[15] -
landmarks[16])
    angle_list.append(angle_)
    # pink 小拇指
    angle_ = vector_2d_angle(landmarks[0] - landmarks[18], landmarks[19] -
landmarks[20])
    angle_list.append(angle_)
    angle_list = [abs(a) for a in angle_list]
    return angle_list


def h_gesture(angle_list):
    """
    Determine the hand gesture through 2D features
    :param angle_list: Bending angles of each finger
    :return: Gesture name string
    """
    thr_angle, thr_angle_thumb, thr_angle_s = 65.0, 53.0, 49.0
    if (angle_list[0] < thr_angle_s) and (angle_list[1] < thr_angle_s) and
(angle_list[2] < thr_angle_s) and (
            angle_list[3] < thr_angle_s) and (angle_list[4] < thr_angle_s):
        gesture_str = "five"
    elif (angle_list[0] > 5) and (angle_list[1] < thr_angle_s) and
(angle_list[2] > thr_angle) and (
            angle_list[3] > thr_angle) and (angle_list[4] > thr_angle):
        gesture_str = "one"
    else:
        gesture_str = "none"
    return gesture_str


class State(enum.Enum):
    NULL = 0
    TRACKING = 1
    RUNNING = 2


def draw_points(img, points, tickness=4, color=(255, 0, 0)):
    """
    Draw connected lines from recorded points on the image
    """
    points = np.array(points).astype(dtype=np.int32)
    if len(points) > 2:
        for i, p in enumerate(points):
            if i + 1 >= len(points):
                break
            cv2.line(img, tuple(p), tuple(points[i + 1]), color, tickness)

def get_track_img(points):
    """
    Generate a trajectory image with black background and white lines using
recorded points
    """
    points = np.array(points).astype(dtype=np.int32)
```

```
    x_min, y_min = np.min(points, axis=0).tolist()
    x_max, y_max = np.max(points, axis=0).tolist()
    track_img = np.full([y_max - y_min + 100, x_max - x_min + 100, 1], 0,
dtype=np.uint8)
    points = points - [x_min, y_min]
    points = points + [50, 50]
    draw_points(track_img, points, 1, (255, 255, 255))
    return track_img


class FingerTrajectoryNode(Node):
    def __init__(self):
        super().__init__('finger_trajectory')
        self.drawing = mp.solutions.drawing_utils
        self.timer = time.time()

        self.hand_detector = mp.solutions.hands.Hands(
            static_image_mode=False,
            max_num_hands=1,
            min_tracking_confidence=0.05,
            min_detection_confidence=0.6
        )

        self.fps = FPS()  # fps计算器
        self.state = State.NULL
        self.points = []
        self.start_count = 0
        self.no_finger_timestamp = time.time()

        self.gc_stamp = time.time()
        self.image_queue = queue.Queue(maxsize=1)
        self.bridge = CvBridge()

        # Initialize video capture device
        self.cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        if not self.cap.isOpened():
            self.get_logger().error("Error: Could not open video device.")
            rclpy.shutdown()

    def image_proc(self):
        ret, frame = self.cap.read()
        if not ret:
            self.get_logger().error("Error: Could not read frame from video
device.")
            return

        rgb_image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        rgb_image = cv2.flip(rgb_image, 1) # horizontal flip
        result_image = np.copy(rgb_image)
        result_call = None
        if self.timer <= time.time() and self.state == State.RUNNING:
            self.state = State.NULL
        try:
            results = self.hand_detector.process(rgb_image) if self.state !=
State.RUNNING else None
            if results is not None and results.multi_hand_landmarks:
```

```
                gesture = "none"
                index_finger_tip = [0, 0]
                self.no_finger_timestamp = time.time()  # record current time
for timeout handling
                for hand_landmarks in results.multi_hand_landmarks:
                    self.drawing.draw_landmarks(
                        result_image,
                        hand_landmarks,
                        mp.solutions.hands.HAND_CONNECTIONS)
                    landmarks = get_hand_landmarks(rgb_image,
hand_landmarks.landmark)
                    angle_list = (hand_angle(landmarks))
                    gesture = (h_gesture(angle_list))
                    index_finger_tip = landmarks[8].tolist()

                if self.state == State.NULL:
                    if gesture == "one":  # detect index finger extended alone,
other fingers clenched
                        self.start_count += 1
                        if self.start_count > 20:
                            self.state = State.TRACKING
                            self.points = []
                    else:
                        self.start_count = 0

                elif self.state == State.TRACKING:
                    if gesture == "five": # extend five fingers to end drawing
                        self.state = State.NULL

                        # generate black and white trajectory image
                        track_img = get_track_img(self.points)
                        contours = cv2.findContours(track_img,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[-2]
                        contour = get_area_max_contour(contours, 300)
                        contour = contour[0]
                        # recognize drawn shape based on trajectory image
                        # cv2.fillPoly draws and fills polygons on the image
                        track_img = cv2.fillPoly(track_img, [contour,], (255,
255, 255))
                        for _ in range(3):
                            # erosion function
                            track_img = cv2.erode(track_img,
cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5)))
                            # dilation function
                            track_img = cv2.dilate(track_img,
cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5)))
                        contours = cv2.findContours(track_img,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[-2]
                        contour = get_area_max_contour(contours, 300)
                        contour = contour[0]
                        h, w = track_img.shape[:2]

                        track_img = np.full([h, w, 3], 0, dtype=np.uint8)
                        track_img = cv2.drawContours(track_img, [contour, ], -1,
(0, 255, 0), 2)
                        # perform polygon fitting on image contour points
                        approx = cv2.approxPolyDP(contour, 0.026 *
cv2.arcLength(contour, True), True)
```

```
                        track_img = cv2.drawContours(track_img, [approx, ], -1,
(0, 0, 255), 2)

                        print(len(approx))
                        # determine shape based on number of vertices in contour
envelope
                        if len(approx) == 3:
                            cv2.putText(track_img, 'Triangle', (10,
40),cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 255, 0), 2)
                        if len(approx) == 4 or len(approx) == 5:
                            cv2.putText(track_img, 'Square', (10,
40),cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 255, 0), 2)
                        if 5 < len(approx) < 10:
                            cv2.putText(track_img, 'Circle', (10,
40),cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 255, 0), 2)
                        if len(approx) == 10:
                            cv2.putText(track_img, 'Star', (10,
40),cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 255, 0), 2)

                        cv2.imshow('track', track_img)

                    else:
                        if len(self.points) > 0:
                            if distance(self.points[-1], index_finger_tip) > 5:
                                self.points.append(index_finger_tip)
                        else:
                            self.points.append(index_finger_tip)

                    draw_points(result_image, self.points)
                else:
                    pass
            else:
                if self.state == State.TRACKING:
                    if time.time() - self.no_finger_timestamp > 2:
                        self.state = State.NULL
                        self.points = []

        except BaseException as e:
            self.get_logger().error("e = {}".format(e))

        self.fps.update_fps()
        self.fps.show_fps(result_image)
        result_image = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)
        cv2.imshow('image', result_image)
        key = cv2.waitKey(1)

        if key == ord(' '): # press space to clear recorded trajectory
            self.points = []
        if time.time() > self.gc_stamp:
            self.gc_stamp = time.time() + 1
            gc.collect()


def main(args=None):
    rclpy.init(args=args)
    finger_track_node = FingerTrajectoryNode()
    try:
        while rclpy.ok():
```

```
            finger_track_node.image_proc()
    except KeyboardInterrupt:
        pass
    finally:
        finger_track_node.cap.release()
        cv2.destroyAllWindows()
        rclpy.shutdown()


if __name__ == "__main__":
    main()
```